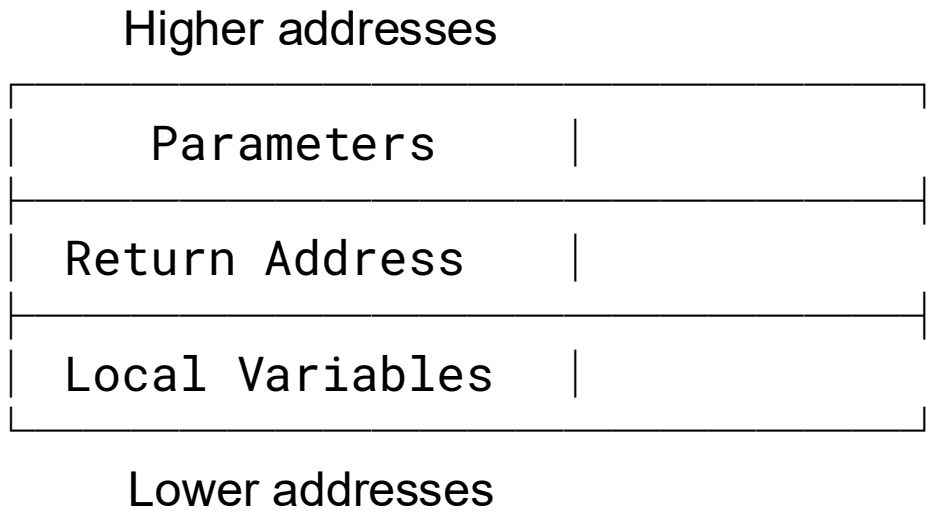


Pointers and Functions

COMP211 Connor McMahon

Diagram Convention

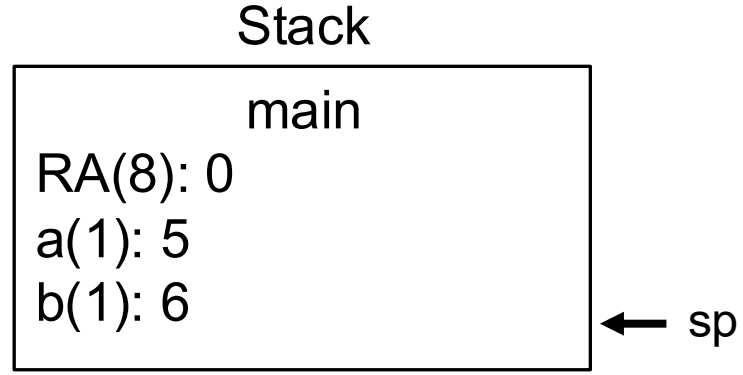
- The caller places the parameters on the stack
- Then the function call places the return address (RA) on the stack.
- The called function then allocates space for its local variables.
- Because the stack grows toward **lower addresses**, the resulting layout is:



Example 1 (Correct Pointer Usage)

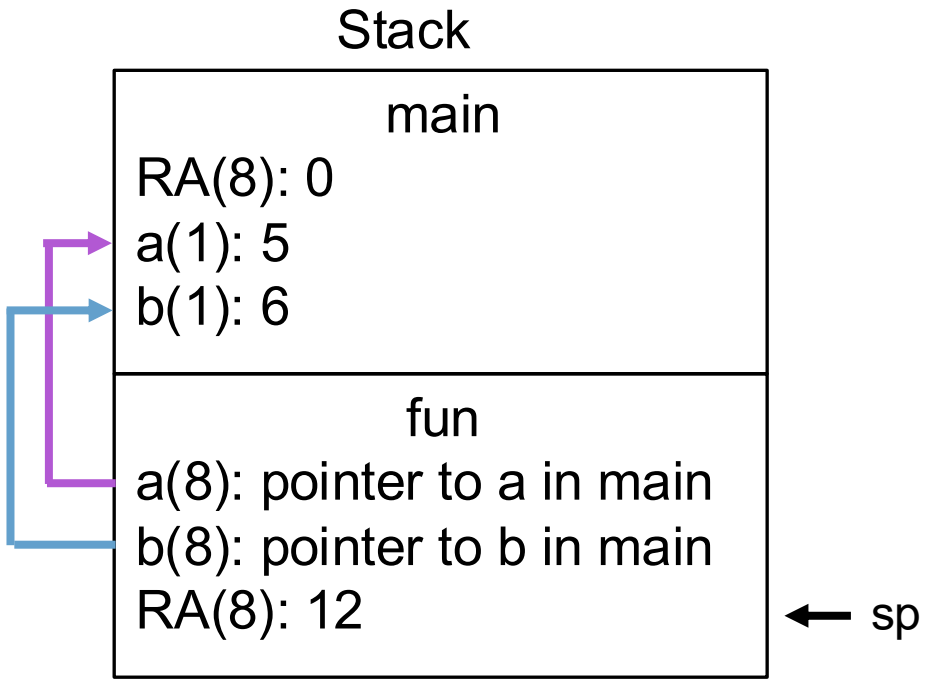
Memory Diagram just before fun is called

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



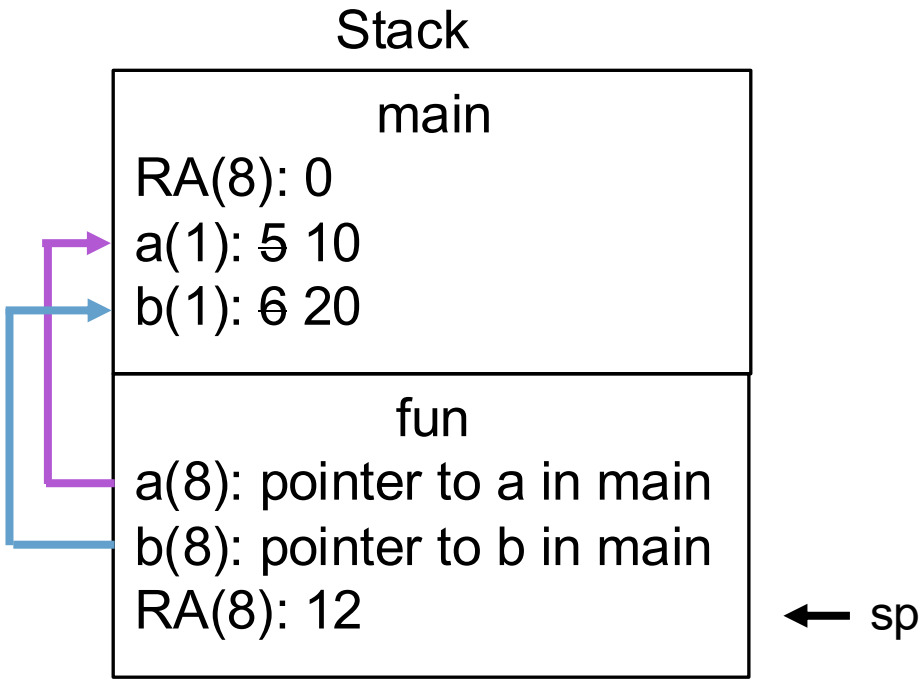
Memory Diagram before line 5 is executed

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



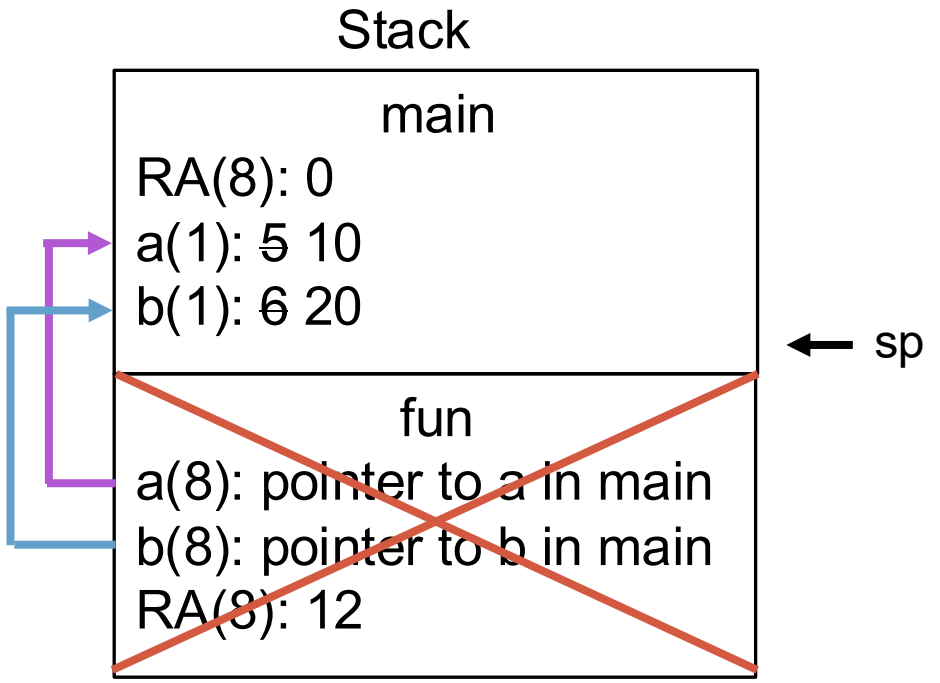
Memory Diagram after line 6 is executed

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



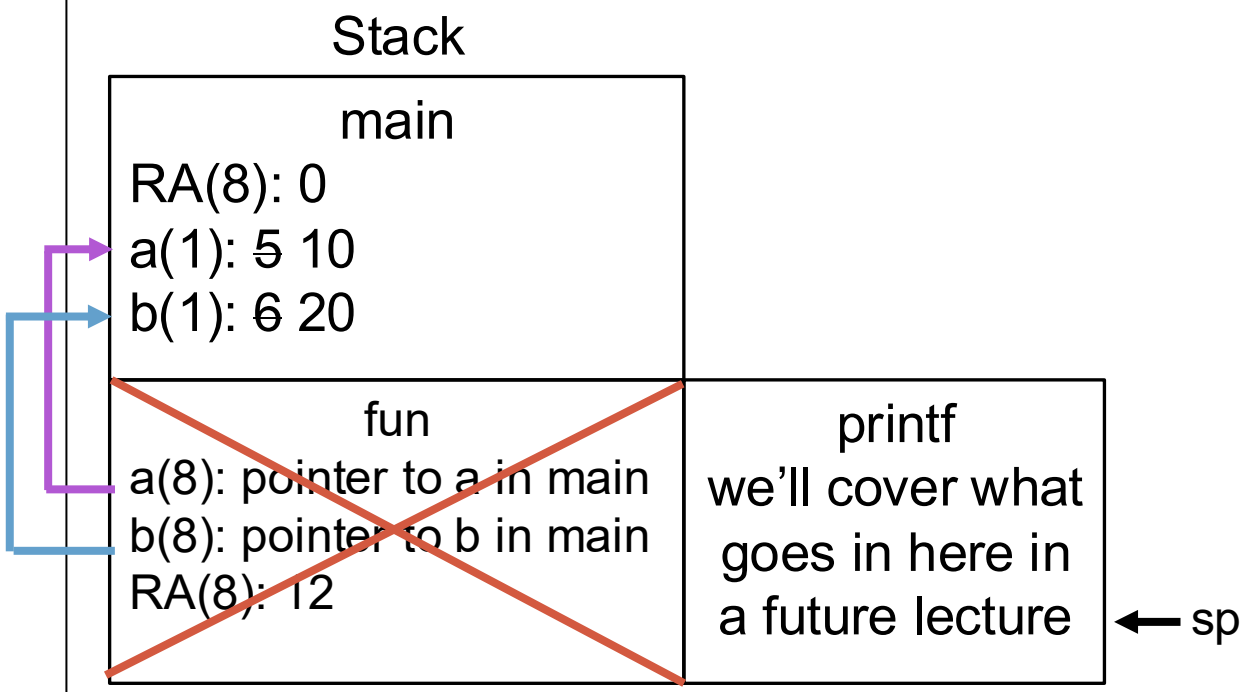
Memory Diagram after fun returns

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



Memory Diagram When printf is called

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```

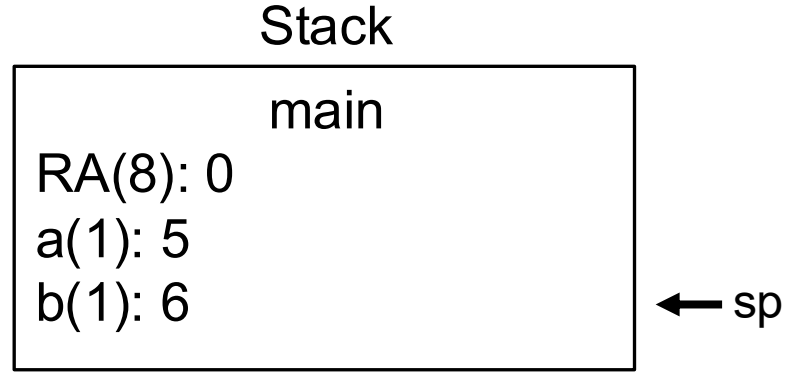


Program Output
a: 10, b: 20

Example 2 (Incorrect Pointer Usage)

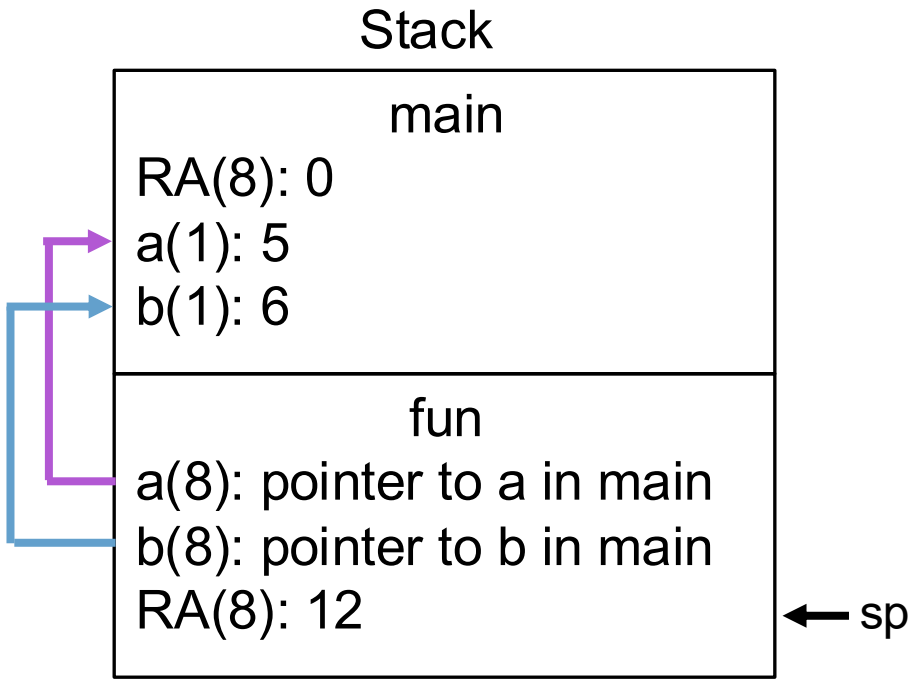
Memory Diagram just before fun is called

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      a = 10;
6      b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



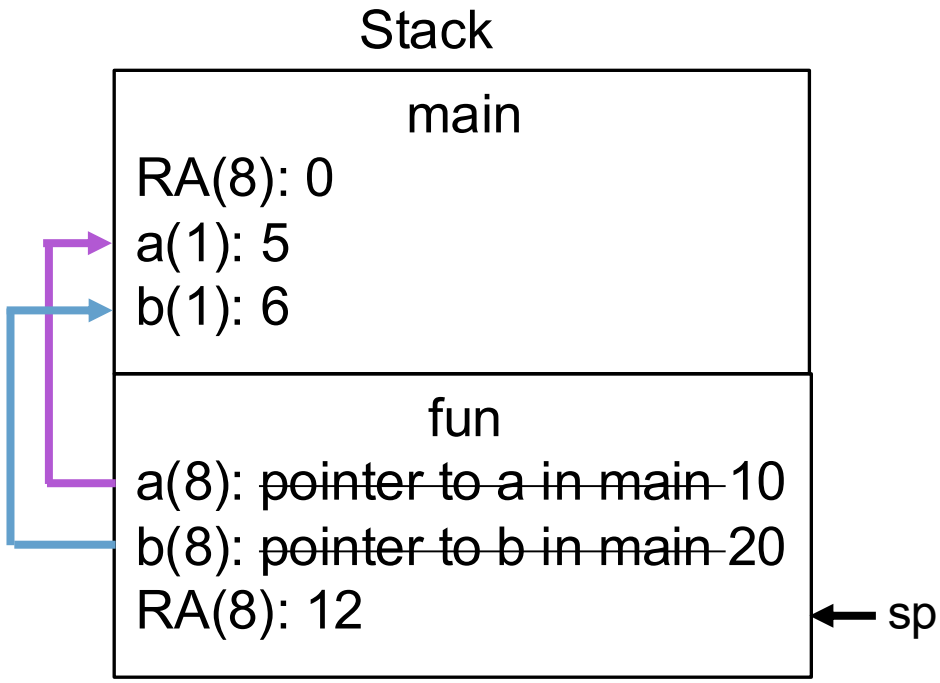
Memory Diagram before line 5 is executed

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      a = 10;
6      b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



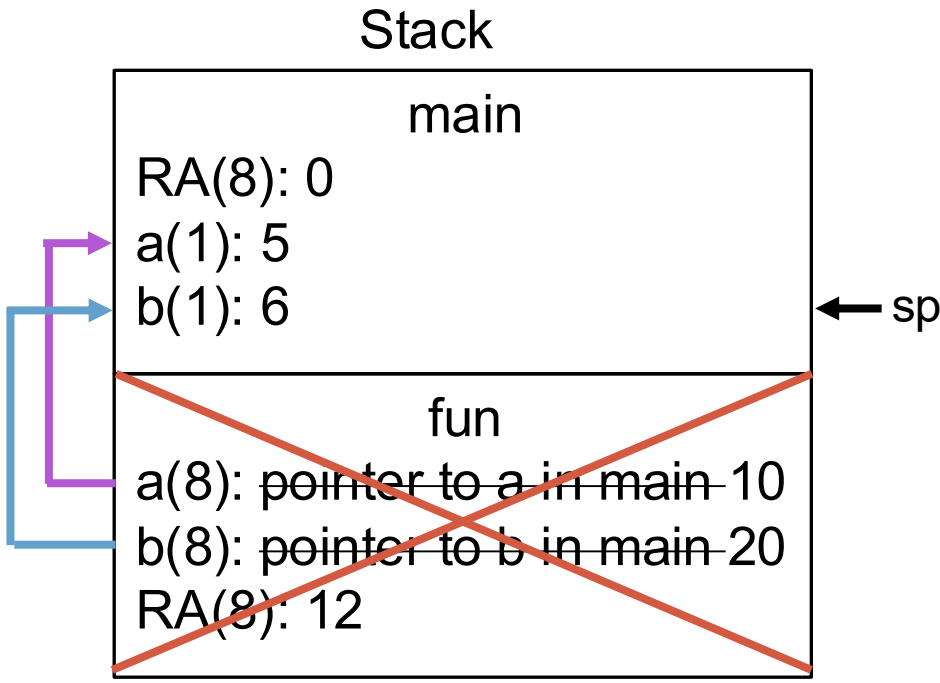
Memory Diagram after line 6 is executed

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      a = 10;
6      b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



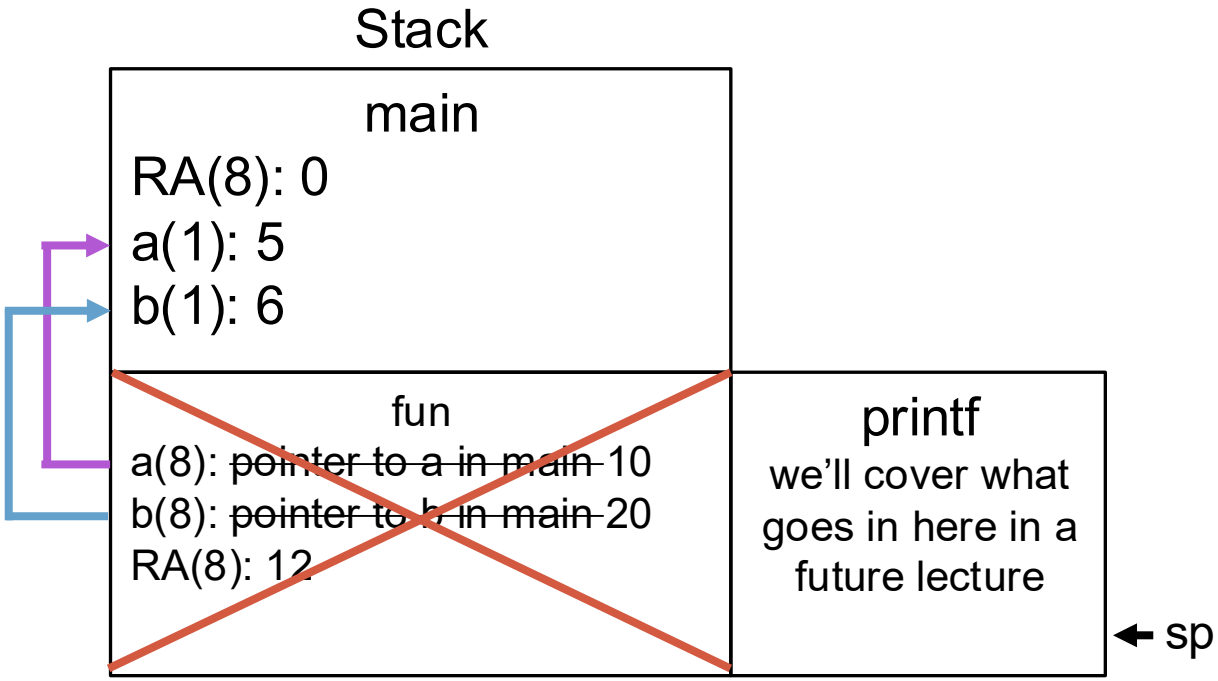
Memory Diagram after fun returns

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      a = 10;
6      b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



Memory Diagram when printf is called

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      a = 10;
6      b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(&a, &b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```

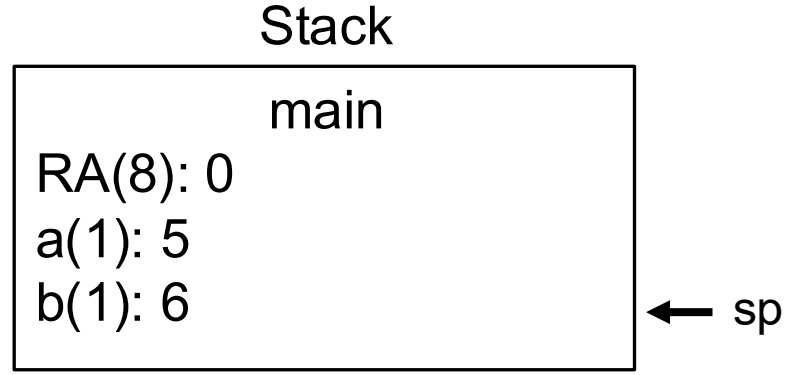


Program Output
a: 5, b: 6

Example 3 (Incorrect Pointer Usage)

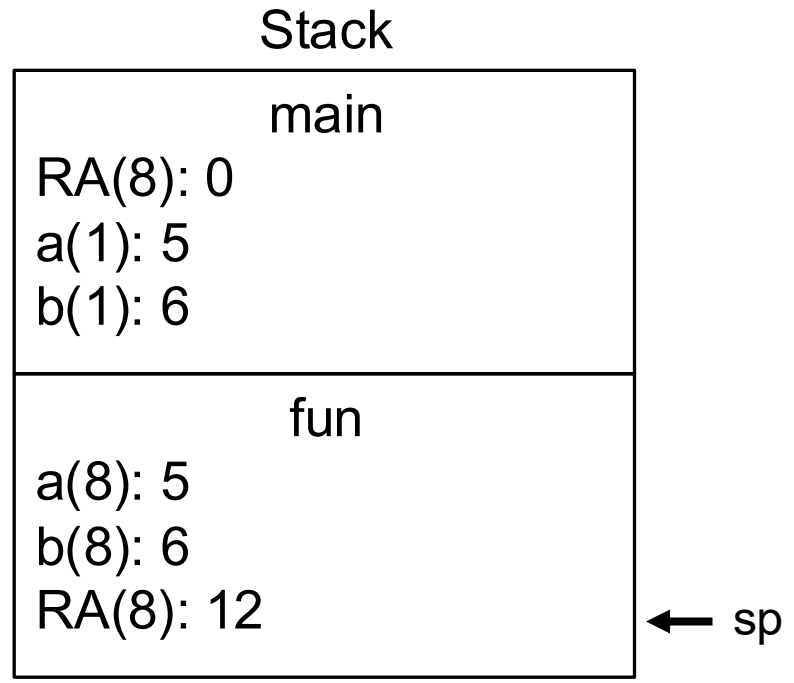
Memory Diagram just before fun is called

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(a, b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



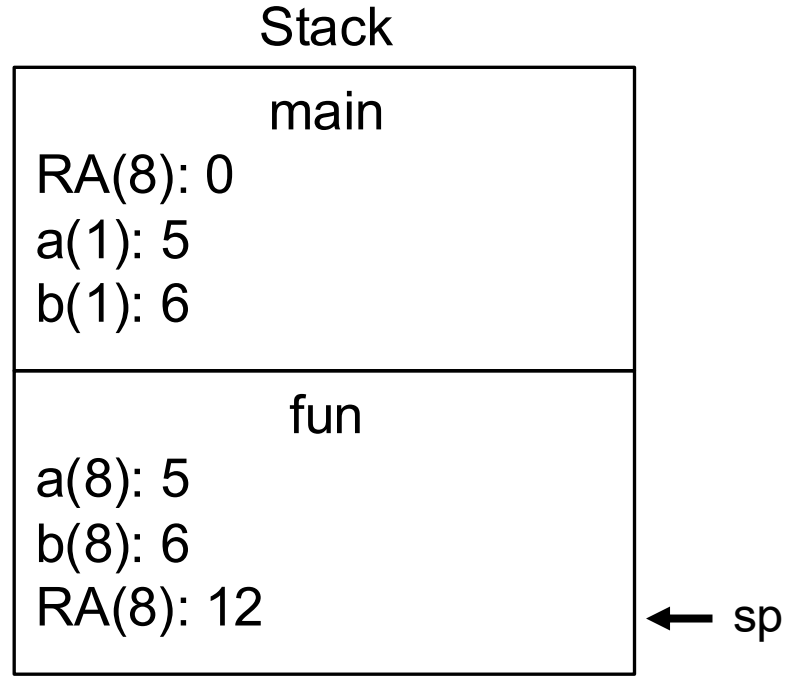
Memory Diagram before line 5 is executed

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(a, b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```



Line 5 Produces a Segmentation Fault

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void fun(int8_t* a, int8_t* b) {
5      *a = 10;
6      *b = 20;
7  }
8
9  int main(void) {
10     int8_t a = 5;
11     int8_t b = 6;
12     fun(a, b);
13     printf("a: %d, b: %d\n", a, b);
14 }
```




```
learncli$ ./a.out
Segmentation fault
```

Segmentation Fault

- A **segmentation fault** (often called a *segfault*) happens when your program tries to access memory it's not allowed to use.

Short Code Writing Examples

addOne

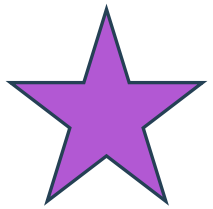
```c
#include <stdio.h>

void addOne(int* n) {


}

int main() {
   int x = 10;
   addOne(&x);
   printf("x = %d\n", x);
   return 0;
}
```

addOne



```c
#include <stdio.h>

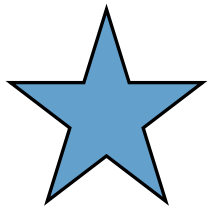
void addOne(int* n) {
   *n = *n + 1;
}

int main() {
   int x = 10;
   addOne(&x);
   printf("x = %d\n", x);
   return 0;
}
```

Complete the implementation of swap

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void swap(int16_t* a, int16_t* b) {
5
6
7
8  }
9
10 int main(void) {
11     int16_t x = 10;
12     int16_t y = 20;
13     printf("x: %d, y: %d\n", x, y);
14     swap(            );
15     printf("x: %d, y: %d\n", x, y);
16 }
```

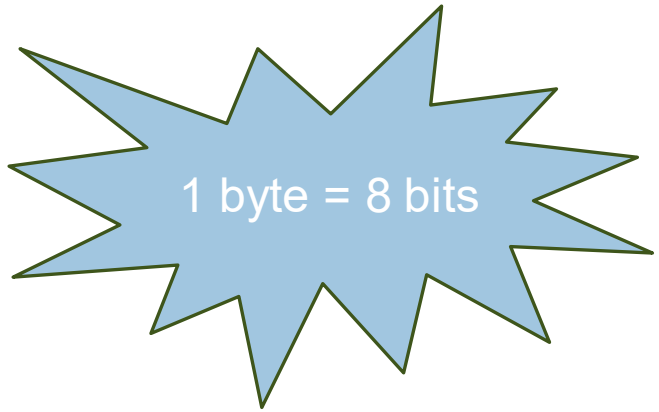
Complete the implementation of swap



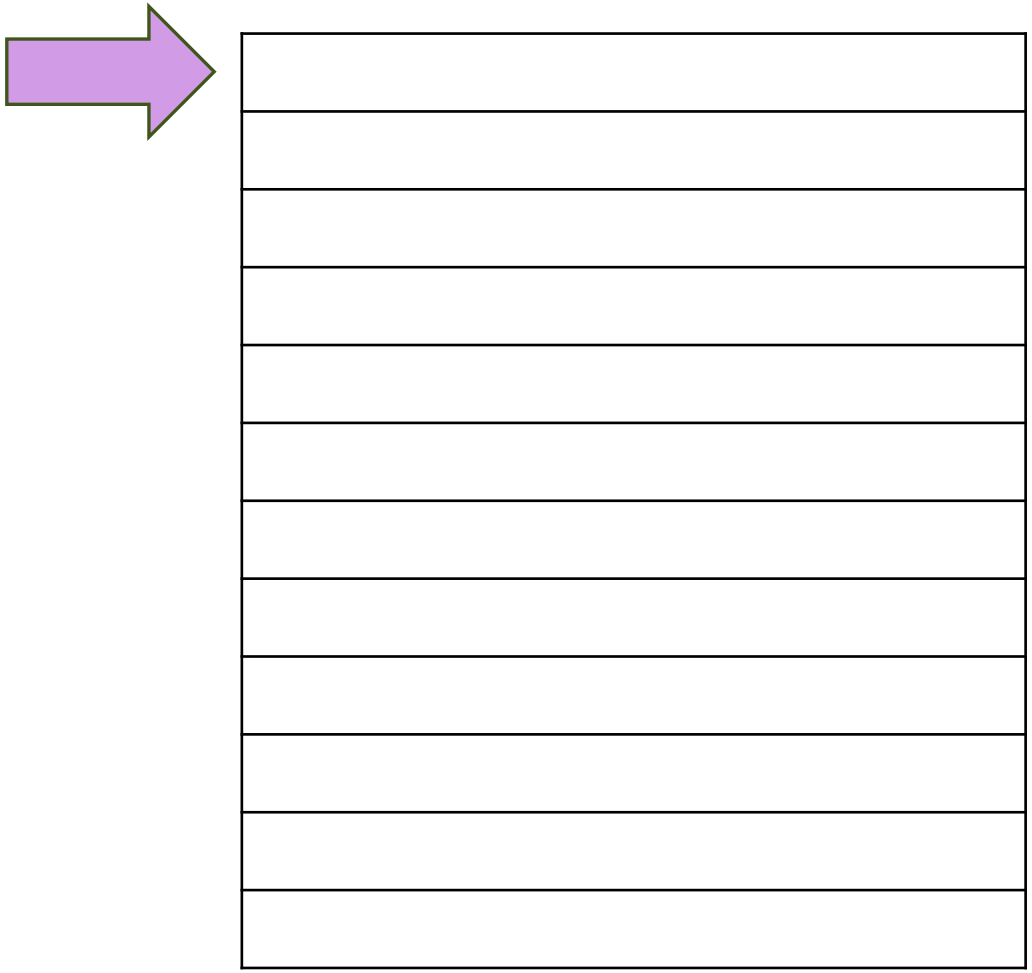
```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void swap(int16_t* a, int16_t* b) {
5      int16_t temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10 int main(void) {
11     int16_t x = 10;
12     int16_t y = 20;
13     printf("x: %d, y: %d\n", x, y);
14     swap(&x, &y);
15     printf("x: %d, y: %d\n", x, y);
16 }
```

Low-Level Memory Diagrams

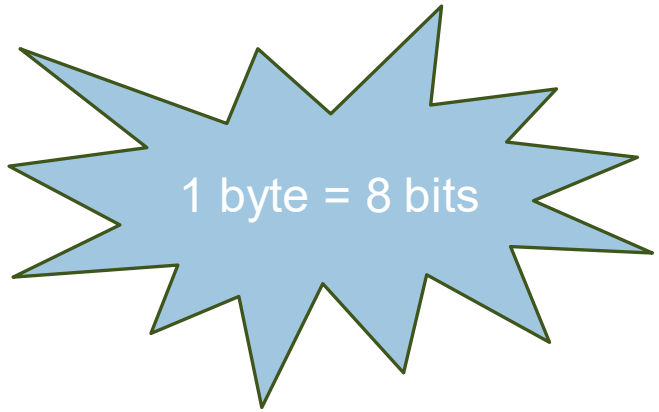
Low-Level Memory Diagram



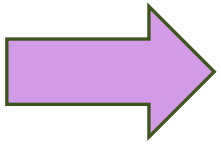
Each of these rows can store one byte of data



Low-Level Memory Diagram



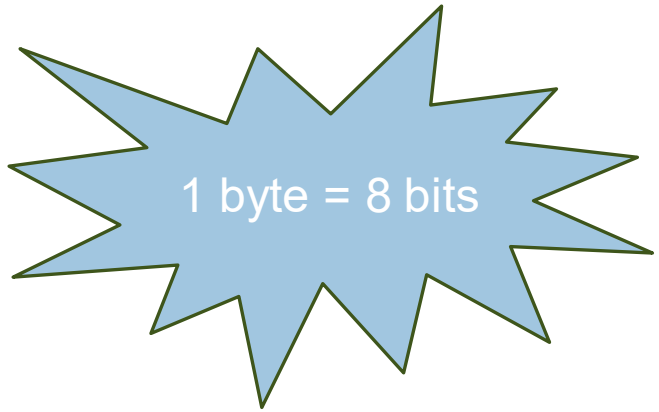
Each of these rows can store one byte of data



0b0000 0101
0b0000 0100
0b1111 1110

Let's say we want to store the number 5 in the top row, 4 in the next row, and -2 in the next row.

Byte Addresses



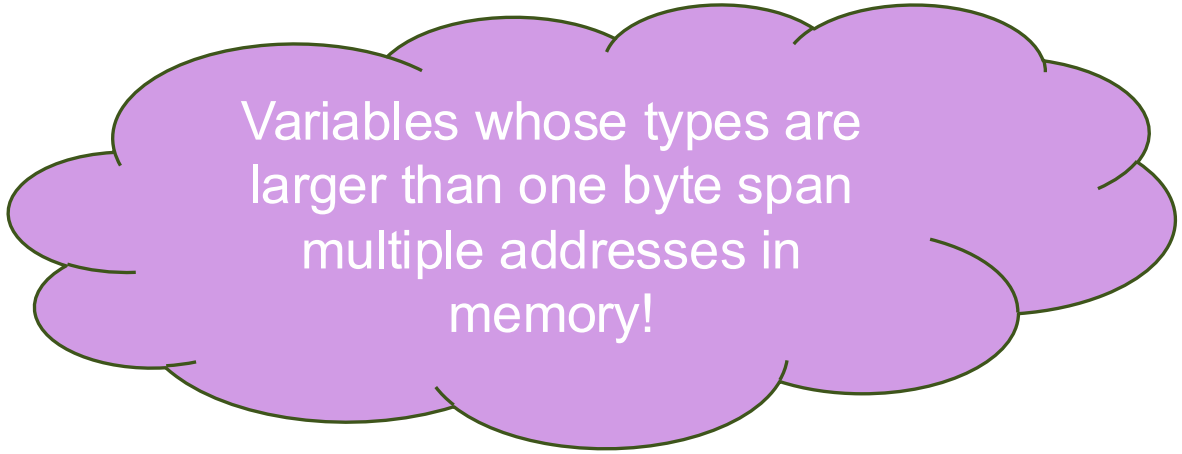
To reference each of these locations, we use something called a byte address.

Note that these addresses are not stored anywhere!

Byte Address	
0	0b0000 0101
1	0b0000 0100
2	0b1111 1110
3	
4	
5	
6	
7	
8	
9	
10	
11	

Storing Larger Data

- What if I want to store an `int32_t`?



Storing Larger Data

- Let's say I want to store an `int32_t` whose value is 329,421

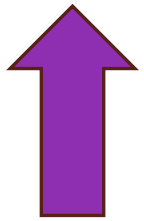
329,421 = 0b 0000 0000 0000 0101 0000 0110 1100 1101

What order should we store the bits?

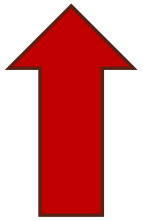


Review: MSB and LSB

0b 0000 0000 0000 0101 0000 0110 1100 1101



Most Significant Byte
(MSB)

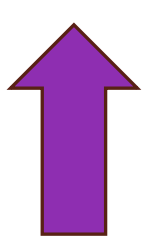


Least Significant Byte
(LSB)

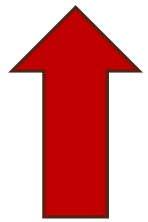
Little Endian Ordering

- The least significant byte of the integer goes in the lowest address

0b 0000 0000 0000 0101 0000 0110 1100 1101



Most Significant Byte
(MSB)



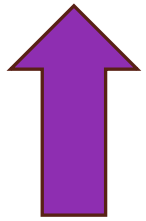
Least Significant Byte
(LSB)



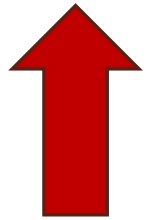
Little Endian Ordering

- The least significant byte of the integer goes in the lowest address

0b 0000 0000 0000 0101 0000 0110 1100 1101



Most Significant Byte
(MSB)



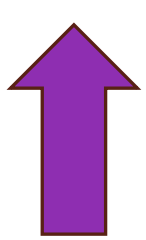
Least Significant Byte
(LSB)

Byte Address	Data
0	1100 1101
1	0000 0110
2	0000 0101
3	0000 0000

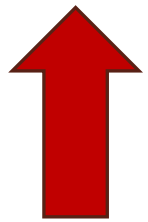
Big Endian Ordering

- The least significant byte of the integer goes in the largest address

0b 0000 0000 0000 0101 0000 0110 1100 1101



Most Significant Byte
(MSB)



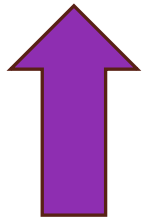
Least Significant Byte
(LSB)



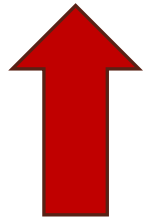
Big Endian Ordering

- The least significant byte of the integer goes in the largest address

0b 0000 0000 0000 0101 0000 0110 1100 1101



Most Significant Byte
(MSB)

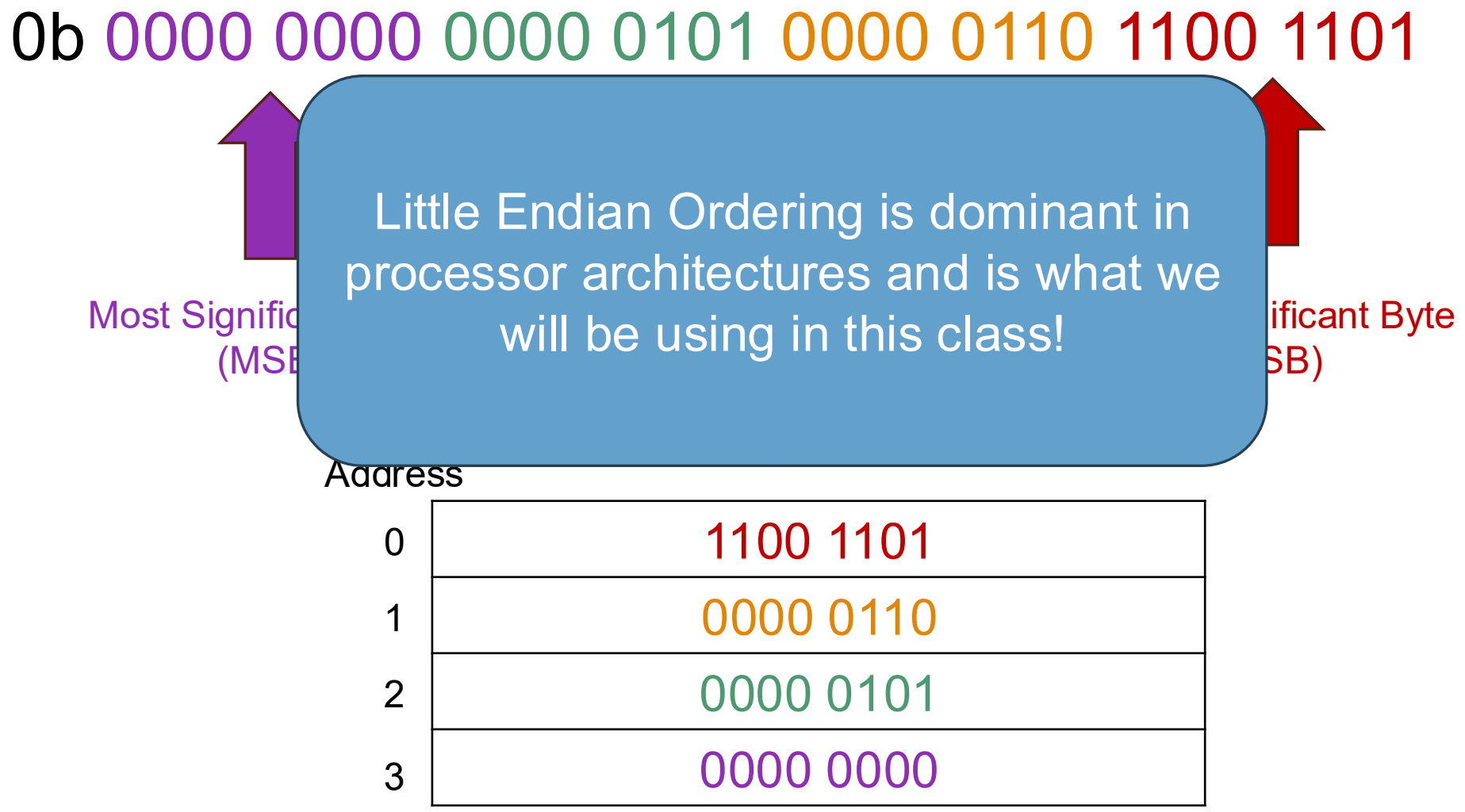


Least Significant Byte
(LSB)

Byte Address	Data
0	0000 0000
1	0000 0101
2	0000 0110
3	1100 1101

Little Endian Ordering

- The least significant byte of the integer goes in the lowest address

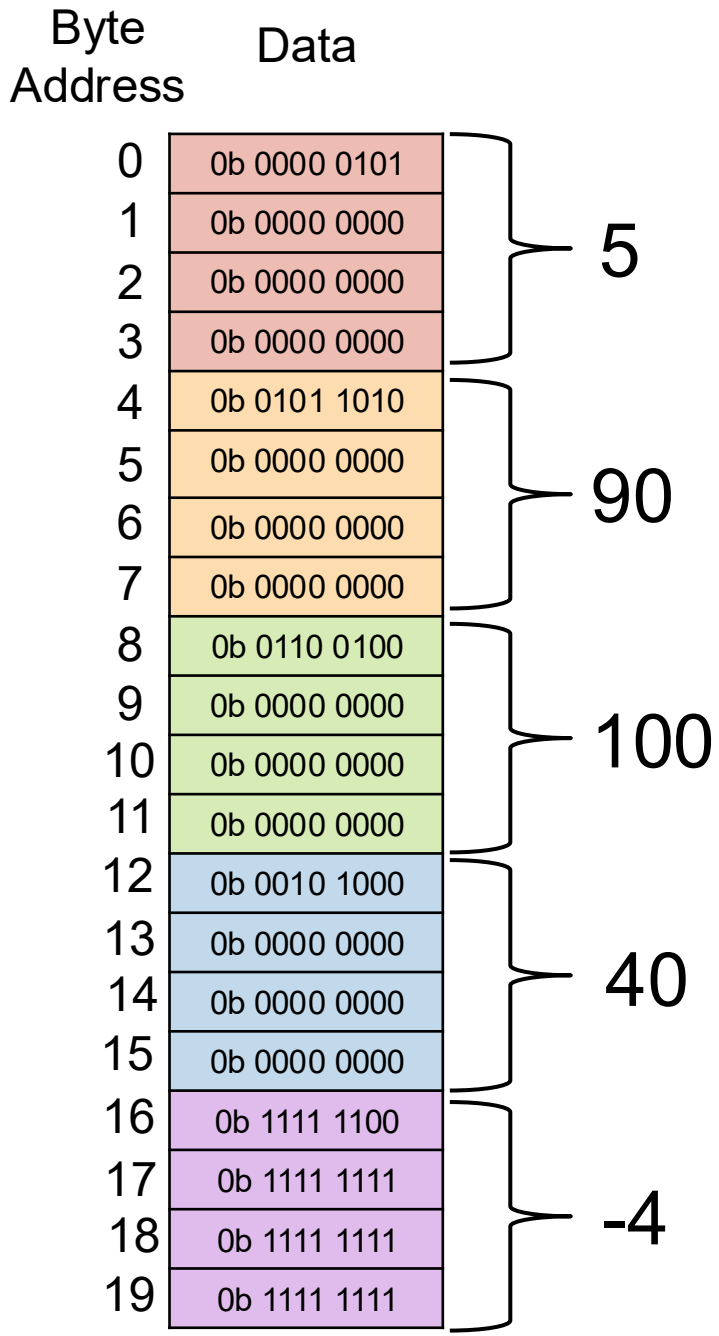


Arrays

- Arrays are stored **contiguously** in memory
  - All the elements of an array are placed in consecutive memory locations.

0	1	2	3	4
5	90	100	40	-4

Example: Array of `int32_t`

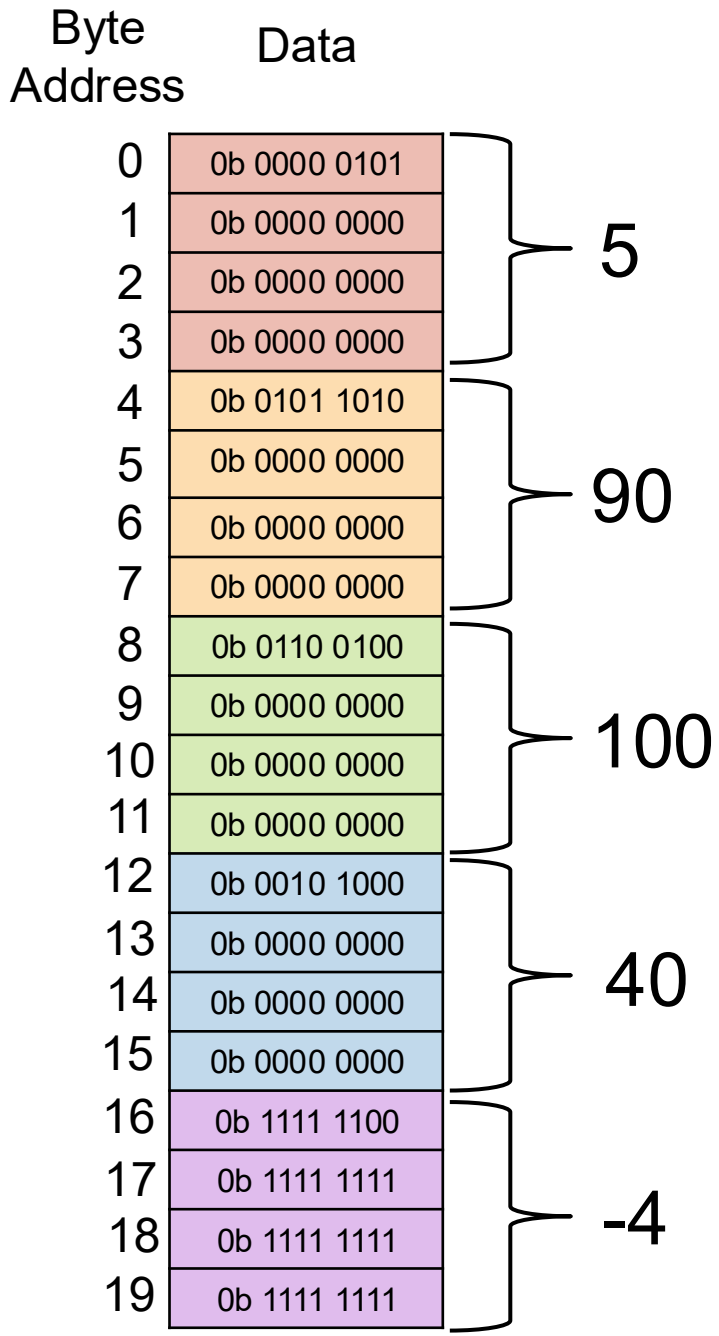


int32_t Array

0	1	2	3	4
5	90	100	40	11

The **address** of each element in the array corresponds to its starting byte in memory.

Index	Value	Address
0	5	
1	90	
2	100	
3	40	
4	11	



int32_t Array

0	1	2	3	4
5	90	100	40	11

The **address** of each element in the array corresponds to its starting byte in memory.

Index	Value	Address
0	5	0
1	90	4
2	100	8
3	40	12
4	11	16

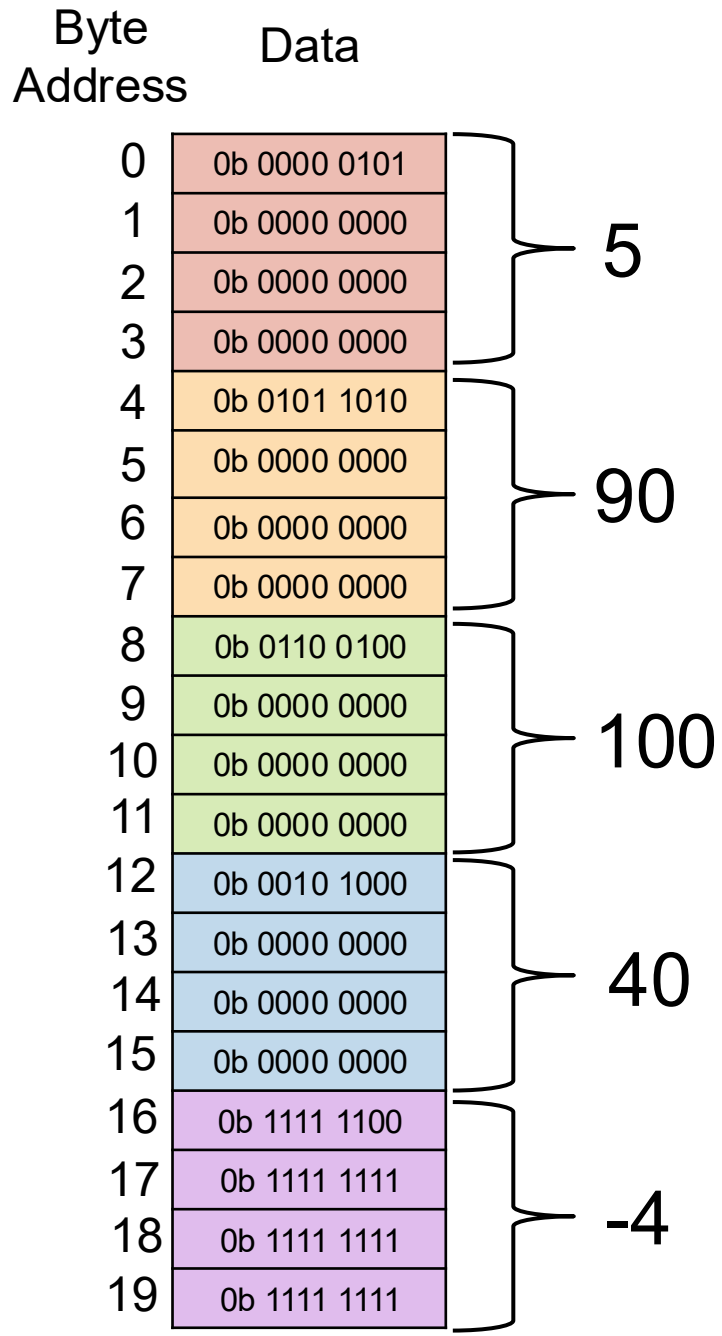


Diagram Configurations

- Sometimes memory diagrams will show 4 bytes per row instead of 1 byte per row
  - See side-by-side comparison on the next slide
- This provides a more compact way to visualize the data
- Much of the data that we are working with is 4 bytes, so this structure makes it easier to see what is stored in memory

One row = one byte

Byte Address	Data	
0	0b 0000 0101	5
1	0b 0000 0000	
2	0b 0000 0000	
3	0b 0000 0000	
4	0b 0101 1010	90
5	0b 0000 0000	
6	0b 0000 0000	
7	0b 0000 0000	
8	0b 0110 0100	100
9	0b 0000 0000	
10	0b 0000 0000	
11	0b 0000 0000	
12	0b 0010 1000	40
13	0b 0000 0000	
14	0b 0000 0000	
15	0b 0000 0000	
16	0b 1111 1100	-4
17	0b 1111 1111	
18	0b 1111 1111	
19	0b 1111 1111	

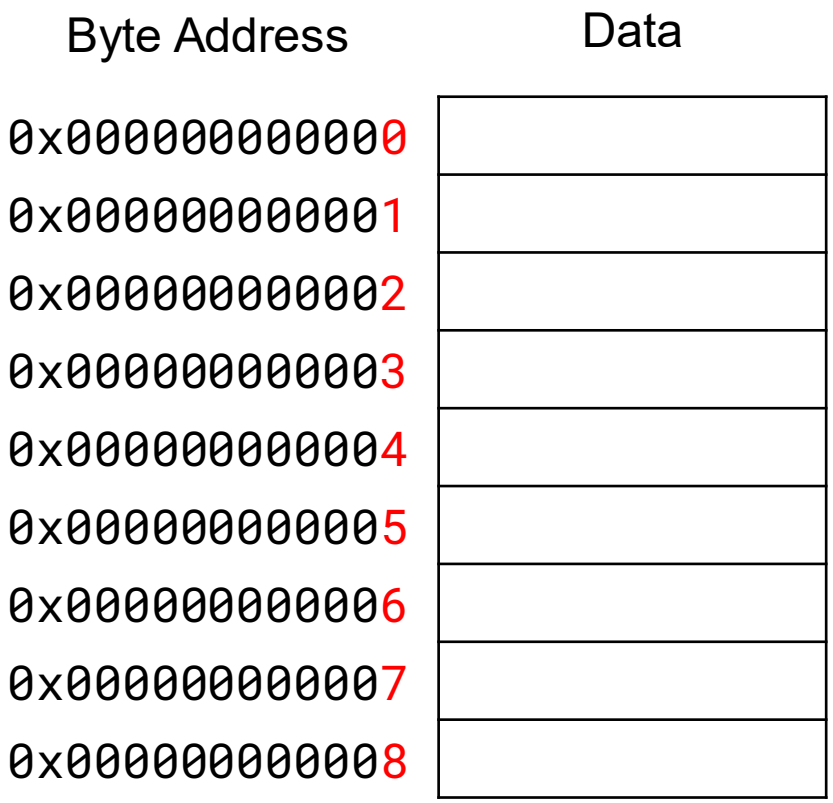
One row = four bytes

Byte Address	Data
0	0b 0000_0000_0000_0000_0000_0000_0000_0101
4	0b 0000_0000_0000_0000_0000_0000_0101_1010
8	0b 0000_0000_0000_0000_0000_0000_0110 0100
12	0b 0000_0000_0000_0000_0000_0000_0010_1000
16	0b 1111_1111_1111_1111_1111_1111_1111_1100

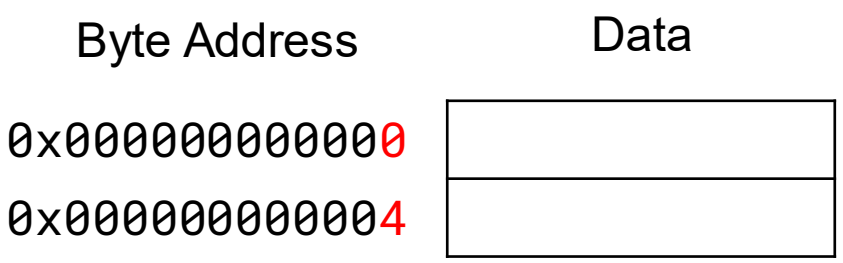
Addresses in Hex

- Addresses are normally written in hex, not decimal
- In this class, we are working with 64-bit addresses
- The rest of the memory diagrams you see in class will look the diagrams on the next slide
- Notice that the compactness of the right diagram allows us to display more data at once than the left diagram

One row = one byte



One row = four bytes



Address Calculation

- **Base address** = address of the start of the array
- Address of any element
  - (base address of array) + (size of each element)(index)

Address Calculation

- I have a 10-element array of `int16_t` with a base address of `0x00000034002C`. What is the byte address of element 9?

Address Calculation

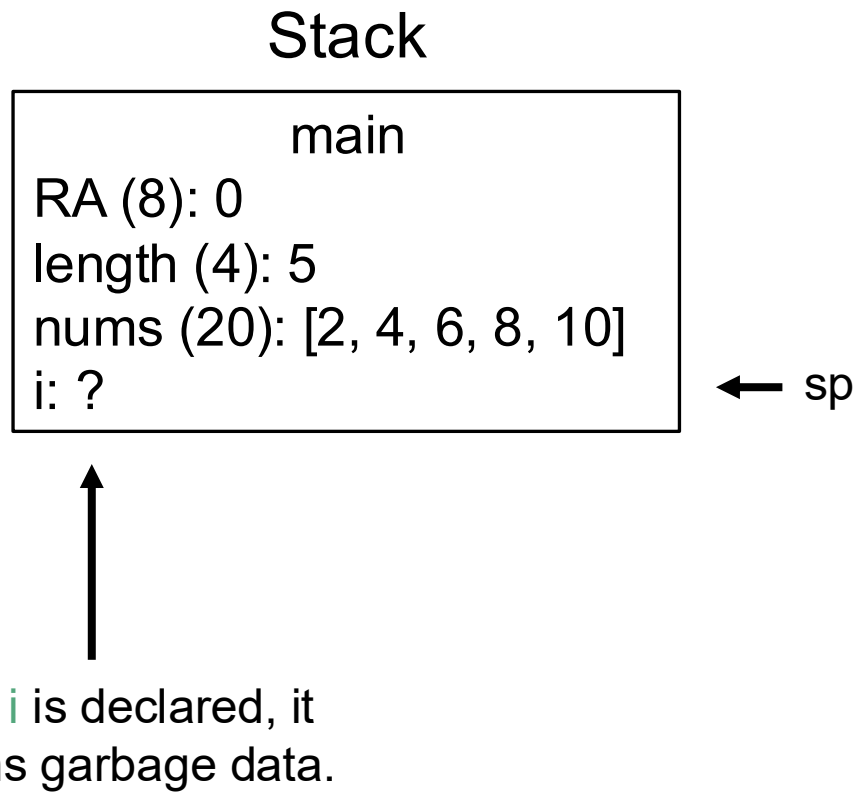


- I have a 10-element array of `int16_t` with a base address of `0x00000034002C`. What is the byte address of element 9?
  - Address of element 9 = base address + sizeof(int16_t) * 9
  - `0x00000034002C + 2(9)`
  - `0x00000034002C + 0x12`
  - `0x00000034003E`

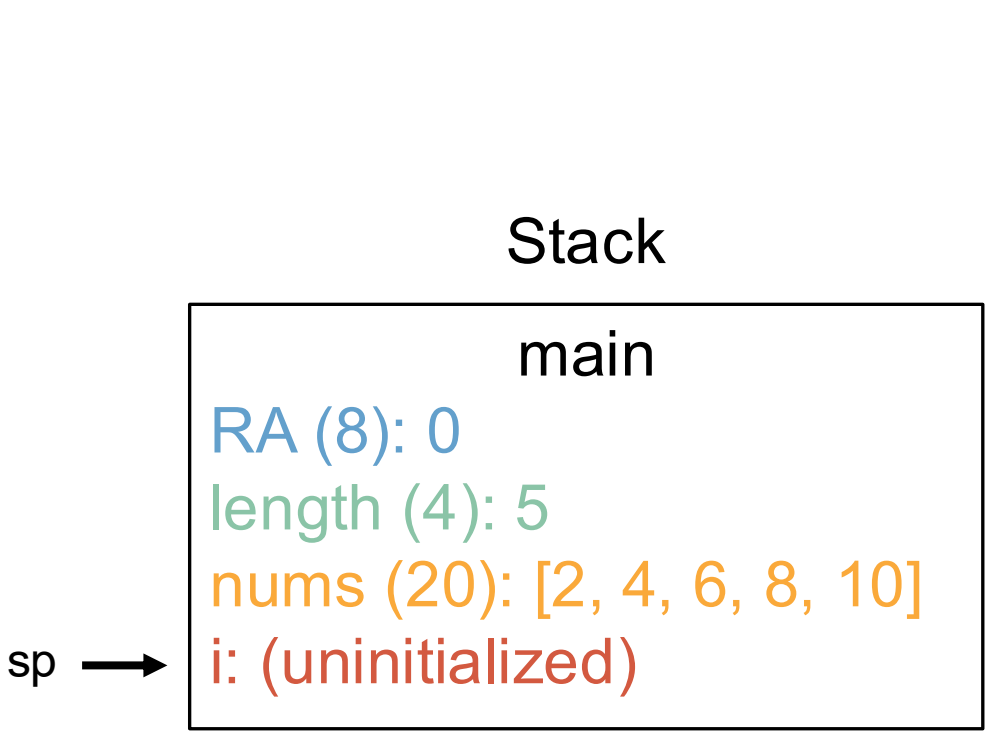
Arrays (High-level Diagram)

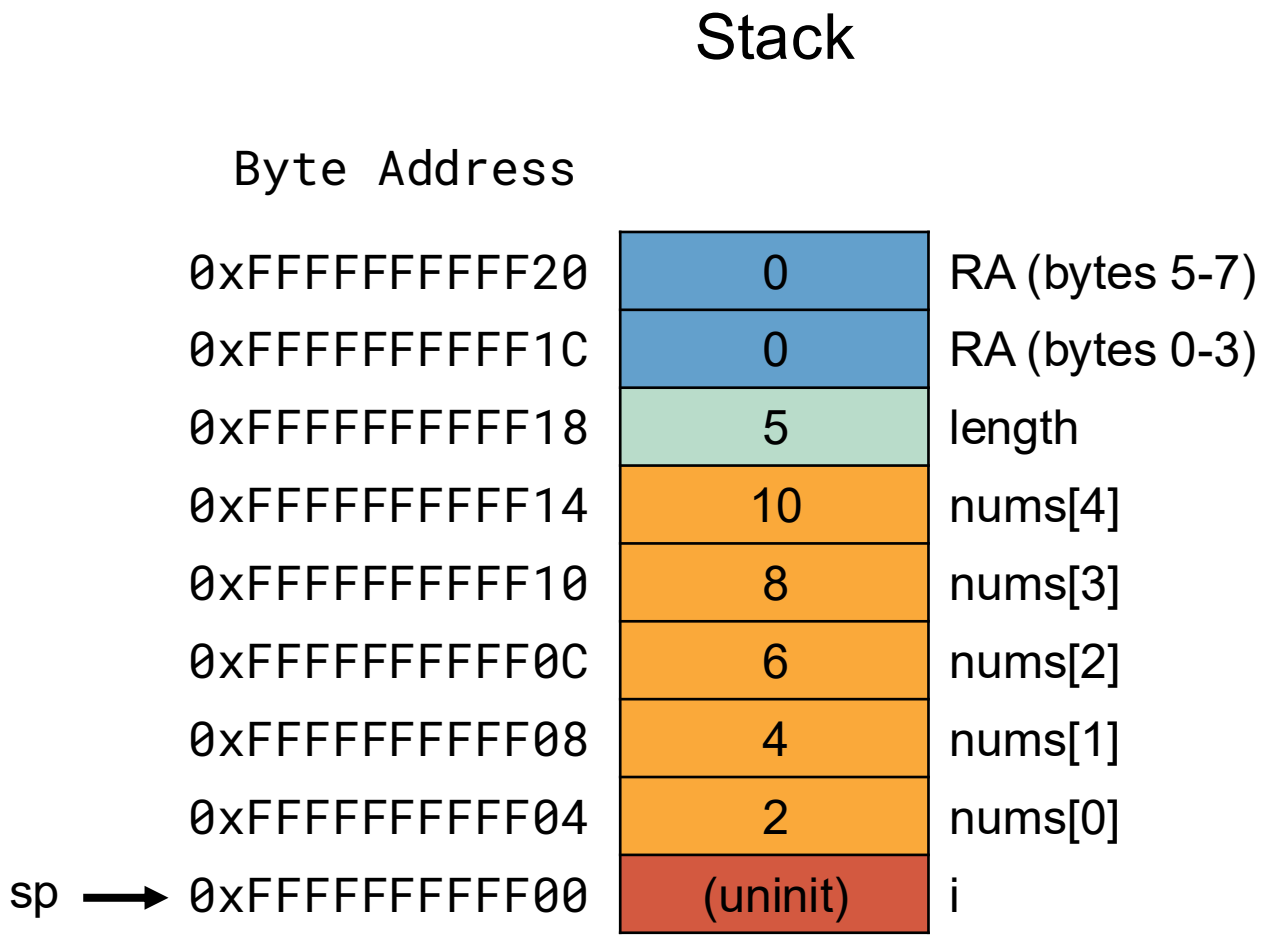
State of the stack just before executing line 8.

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main() {
5          int32_t length = 5;
6          int32_t nums[] = {2, 4, 6, 8, 10};
7          int32_t i;
8          for (i = 0; i < length; i++) {
9                  printf("nums[%d] = %d\n", i, nums[i]);
10         }
11 }
```



Arrays (Low-level Diagram)





Pass-by-value

A copy of the variable is passed

The function works with its own copy

Changes do NOT affect the original variable

Pass-by-pointer

A pointer (address) to the variable is passed

The function accesses the original data

Changes DO affect the original variable

When you pass an array to a function in C, it “decays” to a pointer to its first element.

Pass by Value vs. Pass by Pointer

- Pass by value
  - When `length` is passed to `print_arr()`, a copy of the value is sent
  - Modifying `length` inside `print_arr()` **does not** affect the original variable in `main()`

- Pass by pointer
  - When `nums` is passed to `print_arr()`, instead of copying the entire array, the address of the array is sent
  - Modifying `nums` inside `print_arr()` **does** affect the original array in `main()`

Pass by Value vs. Pass by Pointer

```c
#include <stdint.h>
#include <stdio.h>

void modify_values(int8_t arr[], int8_t length) {
        length = 99; // This change DOES NOT affect length in main()
        arr[0] = 99; // This change DOES affect nums[0] in main()
}

int main() {
        int8_t nums[] = {2, 4, 6, 8, 10};
        int8_t length = 5;

        modify_values(nums, length);

        printf("length in main: %d\n", length); // Still 5
        printf("nums[0] in main: %d\n", nums[0]); // Now 99

        return 0;
}
```

Output:

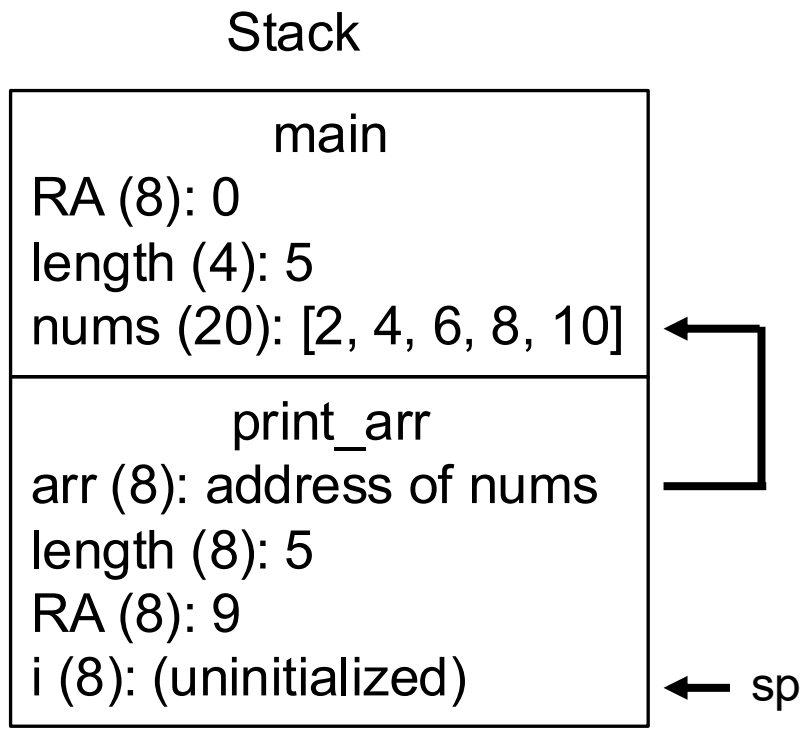
```
length in main: 5
nums[0] in main: 99
```

Pass by Value vs. Pass by Pointer

	Pass-by-Value	Pass-by-Pointer
Pros	• **Safety**: The function cannot modify the original variable. • **Simplicity**: No risk of unintended side effects.	• **Efficient for large data types**: No extra memory allocation for copies. • **Allows modification**: The function can directly modify the original data.
Cons	• **Performance overhead**: Copying large objects is expensive in terms of time and memory. • **More memory usage**: Each call creates a new copy of the variable.	• **Risk of unintended modification**: The function can accidentally change the original variable.

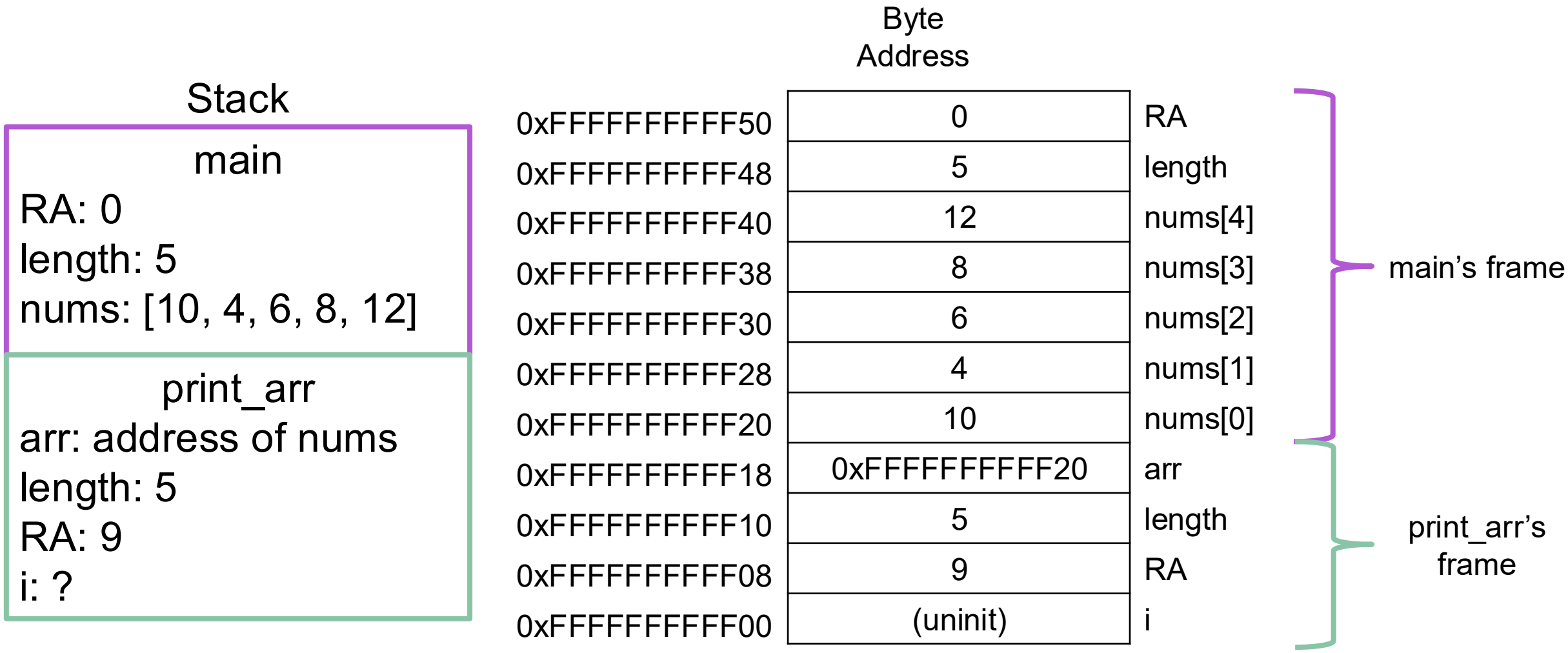
Arrays as Arguments (High-level Diagram)

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  void print_arr(int64_t arr[], int64_t length);
5
6  int main() {
7      int64_t length = 5;
8      int64_t nums[] = {2, 4, 6, 8, 10};
9      print_arr(nums, length);
10 }
11
12 void print_arr(int64_t arr[], int64_t length) {
13     int64_t i;
14     for (i = 0; i < length; i++) {
15         printf("arr[%d] = %d\n", i, arr[i]);
16     }
17 }
```



State of stack before executing line 14

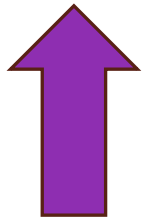
Arrays as Arguments (Low-level Diagram)



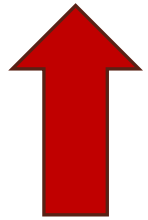
Recall: Little Endian Ordering

- The least significant byte of the integer goes in the lowest address

0b 0000 0000 0000 0101 0000 0110 1100 1101



Most Significant Byte
(MSB)

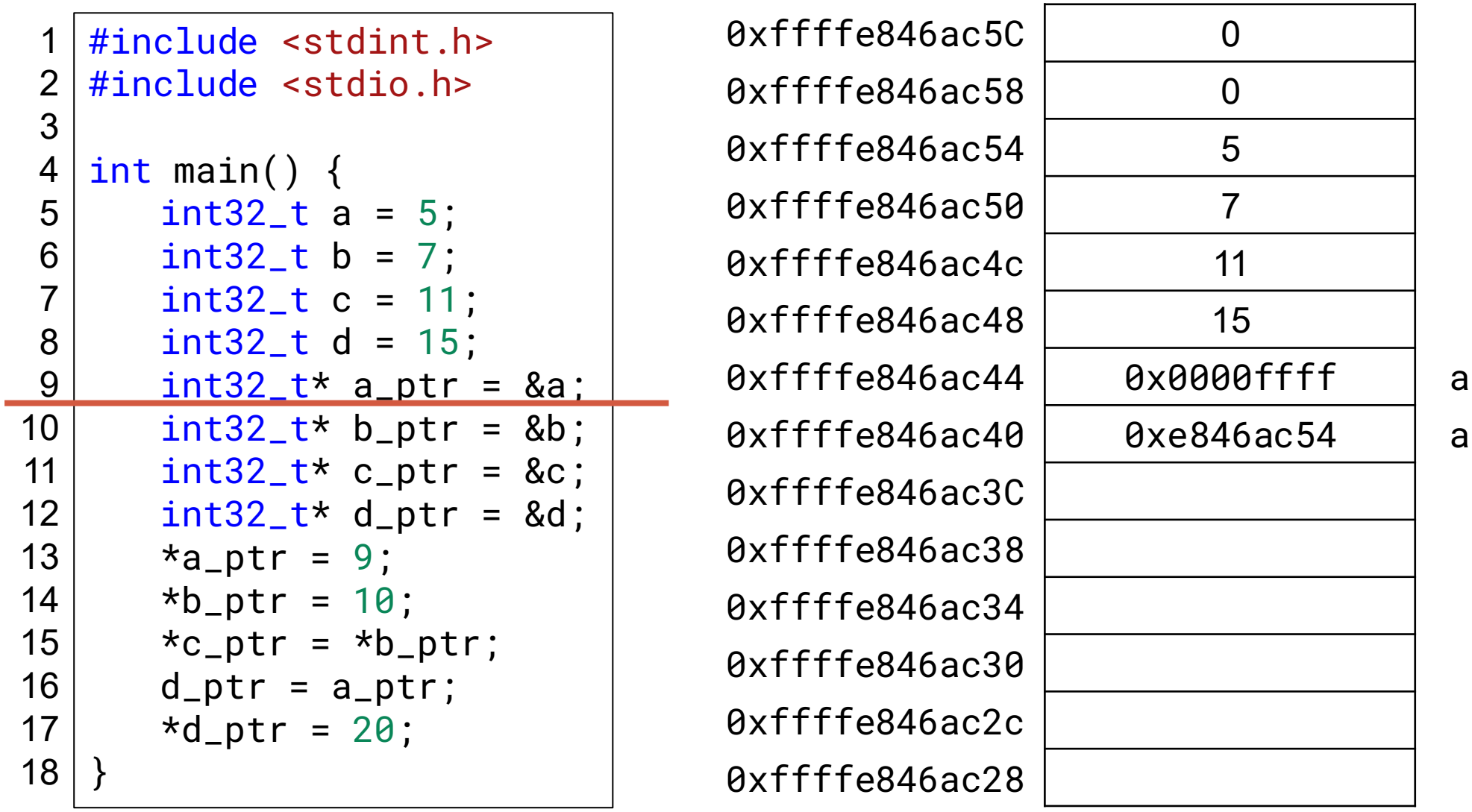


Least Significant Byte
(LSB)

Byte Address	Data
0	1100 1101
1	0000 0110
2	0000 0101
3	0000 0000

Stack

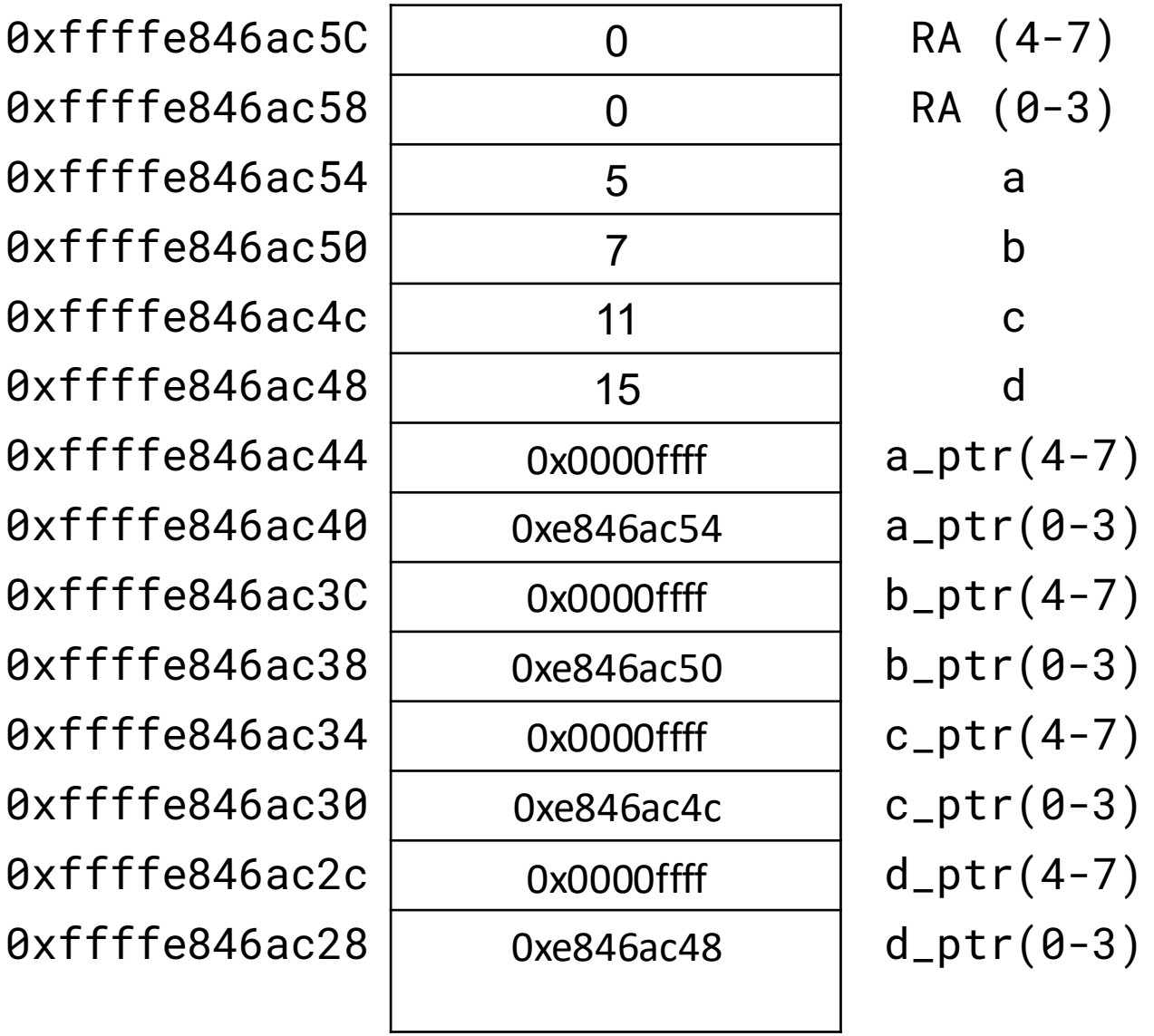
```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main() {
5      int32_t a = 5;
6      int32_t b = 7;
7      int32_t c = 11;
8      int32_t d = 15;
9      int32_t* a_ptr = &a;
10     int32_t* b_ptr = &b;
11     int32_t* c_ptr = &c;
12     int32_t* d_ptr = &d;
13     *a_ptr = 9;
14     *b_ptr = 10;
15     *c_ptr = *b_ptr;
16     d_ptr = a_ptr;
17     *d_ptr = 20;
18 }
```



State after line 12:

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main() {
5      int32_t a = 5;
6      int32_t b = 7;
7      int32_t c = 11;
8      int32_t d = 15;
9      int32_t* a_ptr = &a;
10     int32_t* b_ptr = &b;
11     int32_t* c_ptr = &c;
12     int32_t* d_ptr = &d;
13     *a_ptr = 9;
14     *b_ptr = 10;
15     *c_ptr = *b_ptr;
16     d_ptr = a_ptr;
17     *d_ptr = 20;
18 }
```

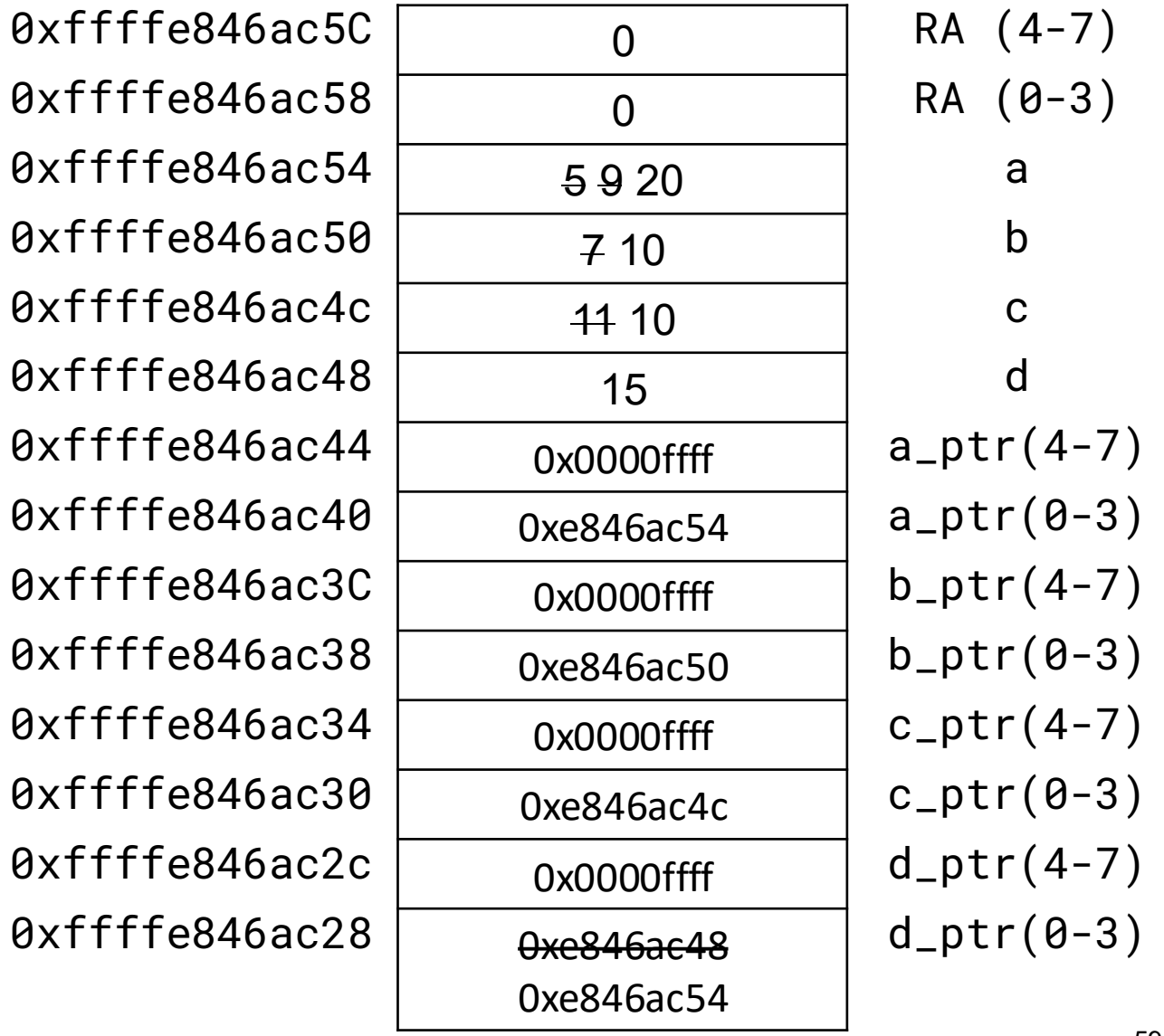
Stack



State after line 17:

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main() {
5      int32_t a = 5;
6      int32_t b = 7;
7      int32_t c = 11;
8      int32_t d = 15;
9      int32_t* a_ptr = &a;
10     int32_t* b_ptr = &b;
11     int32_t* c_ptr = &c;
12     int32_t* d_ptr = &d;
13     *a_ptr = 9;
14     *b_ptr = 10;
15     *c_ptr = *b_ptr;
16     d_ptr = a_ptr;
17     *d_ptr = 20;
18 }
```

Stack



Arrays

- When you use an array name in an expression, it **decays** into:
  - a pointer to the first element
- Pointer arithmetic moves by element size automatically.
  - `arr[i] is the same as *(arr + i)`

```c
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};

    int* p = arr; // same as &arr[0]

    printf("%d\n", *p); // 10
    printf("%d\n", *(p + 1)); // 20
    printf("%d\n", *(p + 2)); // 30

    return 0;
}
```

Pointer Arithmetic

- **Pointer arithmetic** allows you to move between memory locations that a pointer can access.
- It's based on the size of the data type the pointer refers to.

```
int arr[3] = {10, 20, 30};
int *p = arr;
```

When used in an expression, the name of an array (like `arr`) automatically becomes a pointer element 0.

Expression	Meaning	Result
`p`	address of `arr[0]`	(points to 10)
`p + 1`	address of `arr[1]`	(points to 20)
`*(p + 2)`	value at `arr[2]`	30

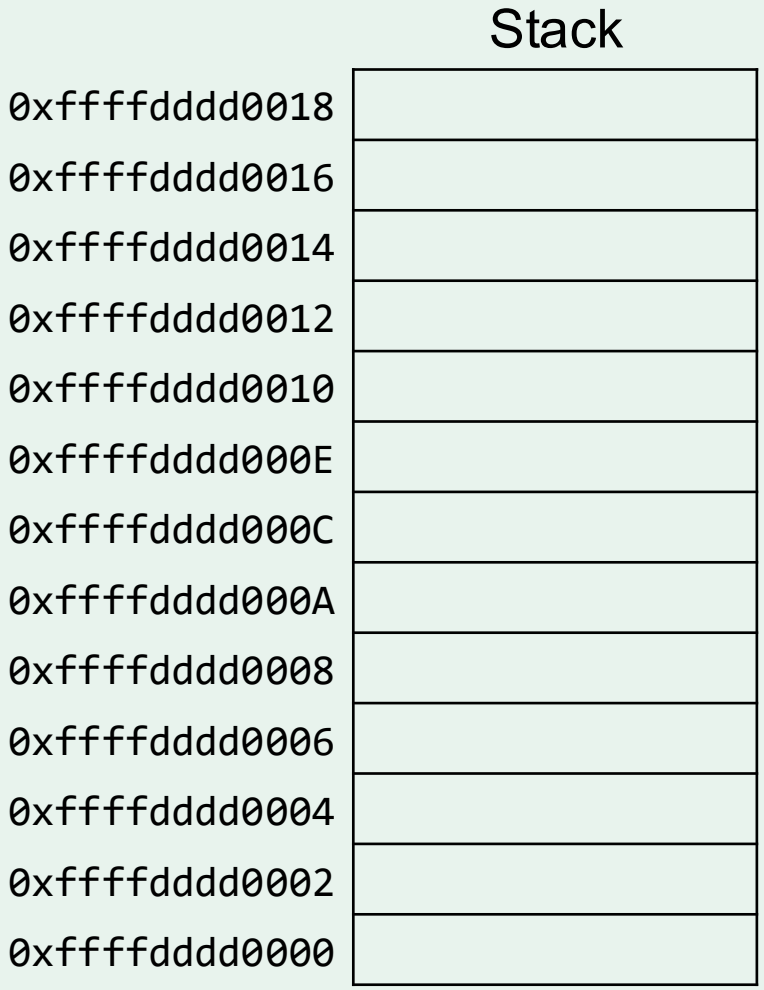
Pointer Arithmetic

- Only valid within the same array (undefined outside).
- Pointer increments depend on data type size
  - (`p + 1` adds `sizeof(*p)` bytes).
- Works with all pointer types (e.g., int*, double*, etc.).

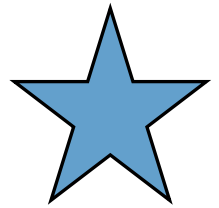
Operation	Description
`p + n`	Move forward `n` elements
`p - n`	Move backward `n` elements
`p1 - p2`	Returns the number of elements between two pointers
`p++` / `p--`	Move to next / previous element

Draw the Low-Level Memory Diagram

```
1  #include <stdio.h>
2
3  int main() {
4          int16_t arr[] = {1, 2, 3, 4, 5};
5          int16_t *ptr = arr;
6
7          *(ptr + 1) = *ptr;
8          *(ptr + 2) = *(ptr + 1);
9
10         return 0;
11 }
```



Draw the Low-Level Memory Diagram



```
1  #include <stdio.h>
2
3  int main() {
4      int16_t arr[] = {1, 2, 3, 4, 5};
5      int16_t *ptr = arr;
6
7      *(ptr + 1) = *ptr;
8      *(ptr + 2) = *(ptr + 1);
9
10     return 0;
11 }
```

Stack

0xffffdddd0018	0	RA (bytes 6-7)
0xffffdddd0016	0	RA (bytes 4-5)
0xffffdddd0014	0	RA (bytes 2-3)
0xffffdddd0012	0	RA (bytes 0-1)
0xffffdddd0010	5	arr[4]
0xffffdddd000E	4	arr[3]
0xffffdddd000C	~~3~~ 1	arr[2]
0xffffdddd000A	~~2~~ 1	arr[1]
0xffffdddd0008	1	arr[0]
0xffffdddd0006	0x0000	ptr (bytes 6-7)
0xffffdddd0004	0xffff	ptr (bytes 4-5)
0xffffdddd0002	0xdddd	ptr (bytes 2-3)
0xffffdddd0000	0x0008	ptr (bytes 0-1)